Python for Artificial Intelligence

CNNs, Lightly

Week 5, Day 3

PROJECT

Today's Project: The Emotion Reader

- Can a neural network look at a small photo of a face and say what emotion it shows? That is today's question
- The data is FER-2013 from Kaggle: about 36,000 grayscale faces, each 48 x 48 pixels, labelled with one of 7 emotions
- This is a hard dataset. Some labels are debatable, and studies report that even people agree with them only about 65 percent of the time, so we will not expect 100%
- Hour 1: how an image becomes numbers, and loading the faces. Hour 2: convolution and pooling, the two ideas that make a CNN a CNN
- Hour 3: build EmotionCNN in TensorFlow, train it, compare it with a dense network, then try ideas to improve it

Hour 1

Images as Numbers

CONCEPT

Why Not Just Use a Dense Network?

- Last week's network took 26 numbers per character. An image is far bigger: our small 48 x 48 face has 2,304 pixels, and a phone photo has millions
- A dense layer connects every pixel to every neuron, so a 1000 x 1000 photo with just 100 hidden neurons needs 100 million weights
- Flattening also throws away the layout: a dense network does not know which pixels are neighbours, so it must relearn the same eye or mouth in every position
- A CNN (Convolutional Neural Network) fixes both problems: it looks at small patches and reuses the same detector everywhere

CONCEPT

Images Are Grids of Numbers

- A grayscale image is a grid of brightness values. In FER-2013, 0 is black and 255 is white
- Colour images stack three grids (red, green, blue), called channels, so a 48 x 48 colour photo is 48 x 48 x 3 numbers
- Our faces are grayscale: 48 x 48 = 2,304 pixels, one channel
- To a computer, a face is just this table of numbers. The network's job is to find patterns in it

A face is a grid of numbers (0 = black, 255 = white)

90	90	90	90	90	90	90	90
90	200	200	200	200	200	200	90
90	200	20	200	200	20	200	90
90	200	45	200	200	45	200	90
90	200	200	150	150	200	200	90
90	200	60	200	200	60	200	90
90	200	200	30	30	200	200	90
90	90	90	90	90	90	90	90

Illustration: 8 x 8 for readability. FER-2013 faces are 48 x 48.

EXAMPLE

Loading FER-2013 in Kaggle

Click Add Input, search for "fer2013", add the dataset, and copy the file path from the panel on the right. Then it is one pd.read_csv, as on Day 20.

```
import numpy as np, pandas as pd

df = pd.read_csv('/kaggle/input/fer2013/fer2013.csv')     # your path may differ
print(df.shape)                          # (35887, 3)
print(df.columns.tolist())               # ['emotion', 'pixels', 'Usage']
print(df['Usage'].value_counts().to_dict())
# {'Training': 28709, 'PublicTest': 3589, 'PrivateTest': 3589}   (the order may vary)
print(len(df['pixels'][0].split()))   # 2304   each image is one long text string of 48 x 48
values
```

CONCEPT

A Look at the Data

- Each row has an emotion (a number 0 to 6), the pixels (one long text string of 2,304 values) and a Usage label that says which split the row belongs to
- Seven emotions: Angry, Disgust, Fear, Happy, Sad, Surprise, Neutral. This is a multi-class problem, with 7 possible answers instead of yes or no
- The classes are uneven: Happy has several thousand faces, Disgust only a few hundred. A model can look decent by ignoring the rare classes
- Some faces are cropped, rotated, partly covered or not faces at all. Real data is messy
- The dataset comes with its own split: Training, PublicTest and PrivateTest. We use them as our train, validation and test sets

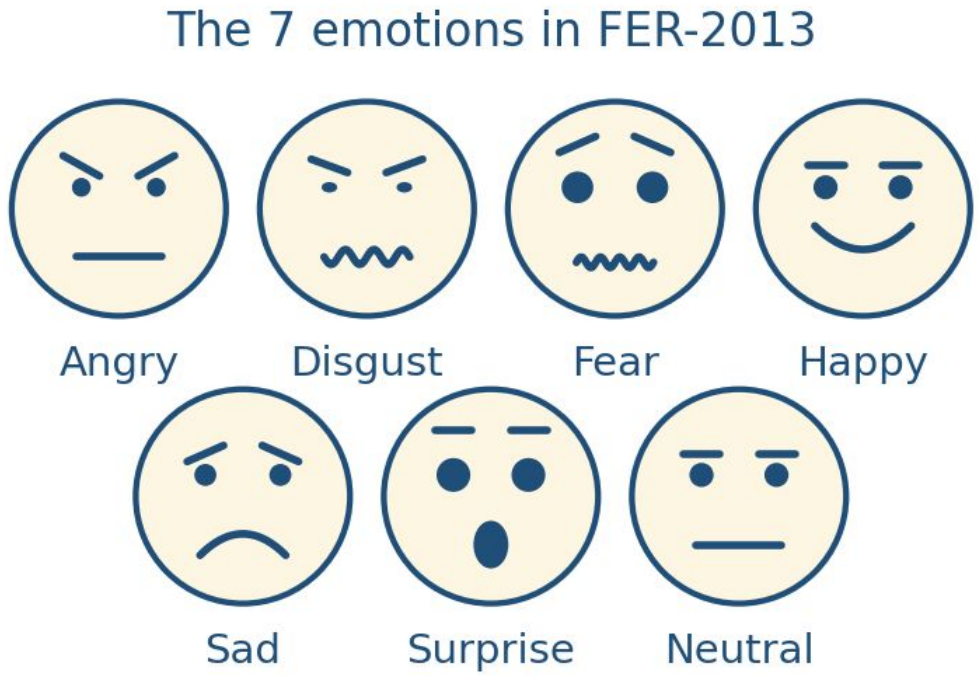


Illustration: simple drawings, not real photos

EXAMPLE

Preparing the Faces

Turn each pixel string into numbers, reshape to 48 x 48 x 1, scale to 0-1, and split using the Usage column.

```
EMOTIONS = ['Angry', 'Disgust', 'Fear', 'Happy', 'Sad', 'Surprise', 'Neutral']

X = np.array([np.array(p.split(), dtype='float32') for p in df['pixels']])   # text -> numbers
X = X.reshape(-1, 48, 48, 1) / 255.0       # 48 x 48 images, 1 channel, scaled to 0-1
y = df['emotion'].values

is_train = (df['Usage'] == 'Training').values       # the dataset's own split
is_val   = (df['Usage'] == 'PublicTest').values
is_test  = (df['Usage'] == 'PrivateTest').values
X_train, y_train = X[is_train], y[is_train]
X_val,   y_val   = X[is_val],   y[is_val]
X_test,  y_test  = X[is_test],  y[is_test]
print(X_train.shape, X_val.shape, X_test.shape)
# (28709, 48, 48, 1) (3589, 48, 48, 1) (3589, 48, 48, 1)   images, height, width, channels
```

EXAMPLE

Look Before You Train

Always look at the data first. This draws one face for each emotion, then counts how many of each there are.

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 7, figsize=(14, 2.4))
for k, ax in enumerate(axes):
    i = np.where(y_train == k)[0][0]                # first training face of emotion k
    ax.imshow(X_train[i].reshape(48, 48), cmap='gray')
    ax.set_title(EMOTIONS[k])
    ax.axis('off')
plt.show()

print(pd.Series(y_train).map(dict(enumerate(EMOTIONS))).value_counts())   # how many of each?
```

QUICK CHECK

Quick Check

How many numbers are needed to store one 48 x 48 colour photo, and how many weights would a dense layer with 100 neurons need to connect to all of them?

ANSWER

Quick Check: Answer

```
# 48 x 48 pixels x 3 colour channels = 6,912 numbers.
#
# A dense layer with 100 neurons needs one weight per input per
# neuron: 6,912 x 100 = 691,200 weights, plus 100 biases.
#
# For our grayscale face it is 2,304 x 100 = 230,400, and that is
# for a tiny image and a single layer. This is why CNNs share weights
# instead of connecting everything to everything.
```

Hour 2

How a CNN Sees

CONCEPT

The Big Idea: Look at Small Patches

- Think of how you read a face: you notice the eyebrows, the shape of the eyes, the curve of the mouth, then combine them into an impression
- A CNN does the same: early layers detect simple patterns such as edges, and later layers combine them into shapes
- The tool that detects a pattern is a small grid of numbers called a filter (or kernel), usually 3 x 3
- The filter slides across the whole image, asking at every position: how much does this patch look like my pattern?

CONCEPT

From Small Patterns to a Decision

A CNN builds its answer in stages. Small detectors fire on simple patterns, later ones combine them, and the last layer decides. (The yes/no questions are an analogy: a real network uses numbers, not words.)

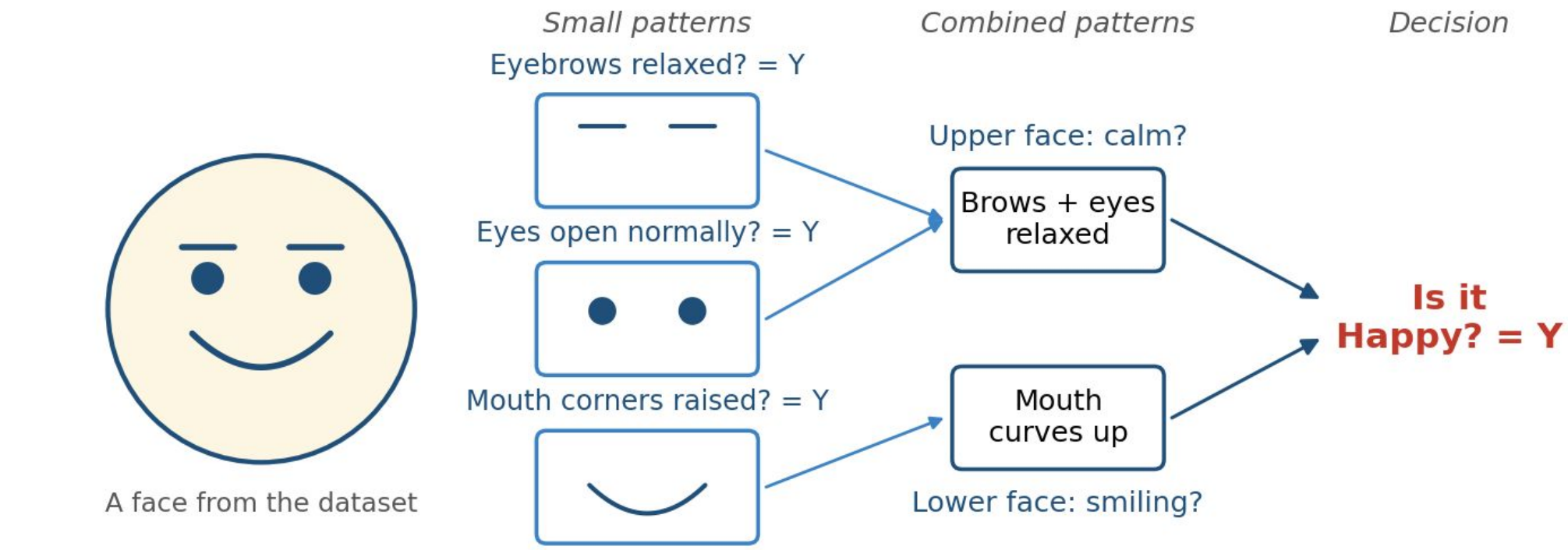
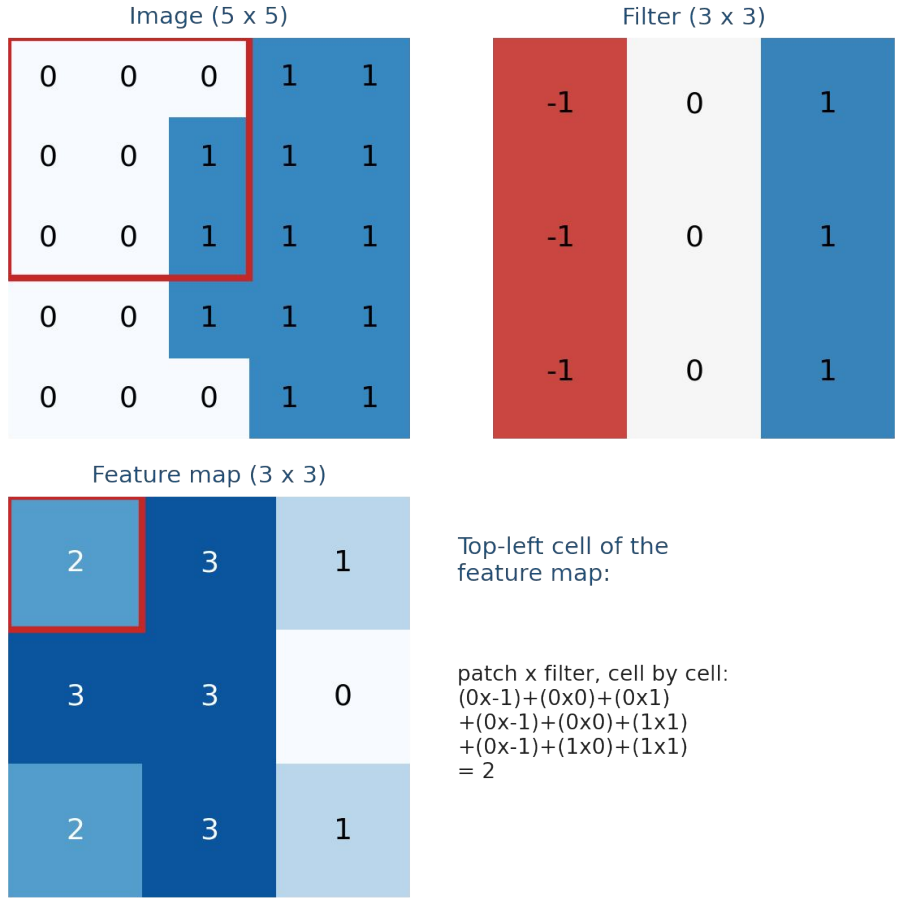


Illustration: simple drawing, not a real photo

CONCEPT

Convolution: Slide a Filter Over the Image

- At each position, multiply the filter with the patch underneath it, cell by cell, and add everything up
- That single number goes into the output grid, called a feature map
- This filter has -1 on the left and +1 on the right, so it lights up where dark meets light, a vertical edge
- The same filter is used at every position, which is called weight sharing and is why CNNs need so few weights



EXAMPLE

Convolution by Hand in NumPy

The same calculation as the diagram: slide a 3 x 3 filter over a 5 x 5 image. The 3 x 3 output is the feature map.

```
import numpy as np

image = np.array([[0,0,0,1,1],
                  [0,0,1,1,1],
                  [0,0,1,1,1],
                  [0,0,1,1,1],
                  [0,0,0,1,1]])
kernel = np.array([[-1,0,1]] * 3)       # vertical-edge filter

out = np.zeros((3, 3), dtype=int)
for i in range(3):
    for j in range(3):
        patch = image[i:i+3, j:j+3]      # the 3 x 3 patch under the filter
        out[i, j] = (patch * kernel).sum()
print(out)    # [[2 3 1] [3 3 0] [2 3 1]]
```

CONCEPT

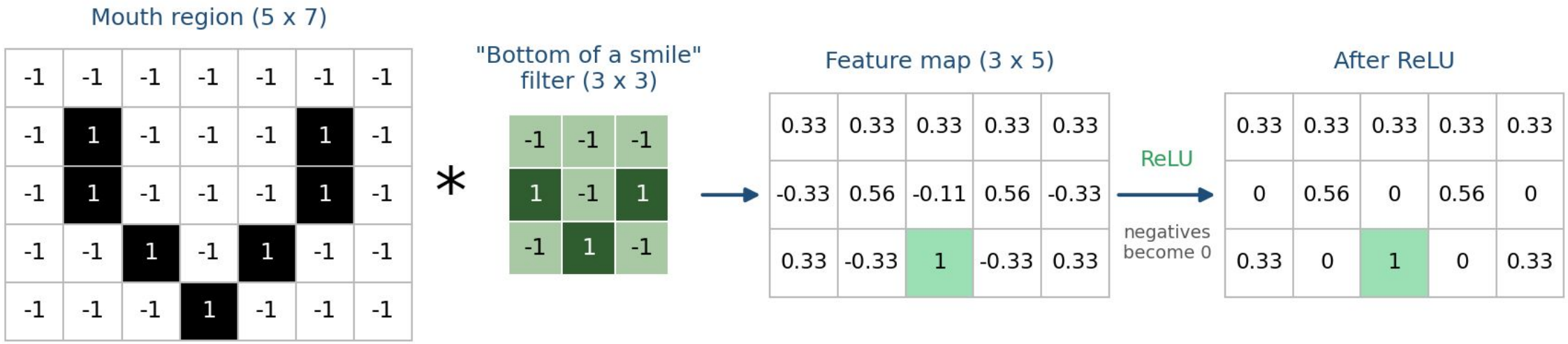
Filters Are Learned, Not Designed

- We just wrote an edge filter by hand. In a CNN nobody designs the filters: the 9 numbers in each 3 x 3 filter are weights, learned by gradient descent just like Day 20
- One filter detects one pattern, so a layer uses several filters at once. Our first layer will use 16, giving 16 feature maps
- Each filter has only 9 weights plus a bias, no matter how large the image is
- After the convolution comes ReLU, exactly as in VillainNet, so the network can learn curved, non-linear patterns

CONCEPT

A Filter for a Smile, Then ReLU

A 3 x 3 filter shaped like the bottom of a smile slides over a tiny mouth region. The feature map is large where the pattern matches, and ReLU sets negative scores to 0.



Value 1 = stroke, -1 = background. The filter responds most (green cell = 1) where its pattern lines up with the smile.
Every number is computed: sum of (patch x filter), divided by 9.

CONCEPT

Padding, Stride and Output Size

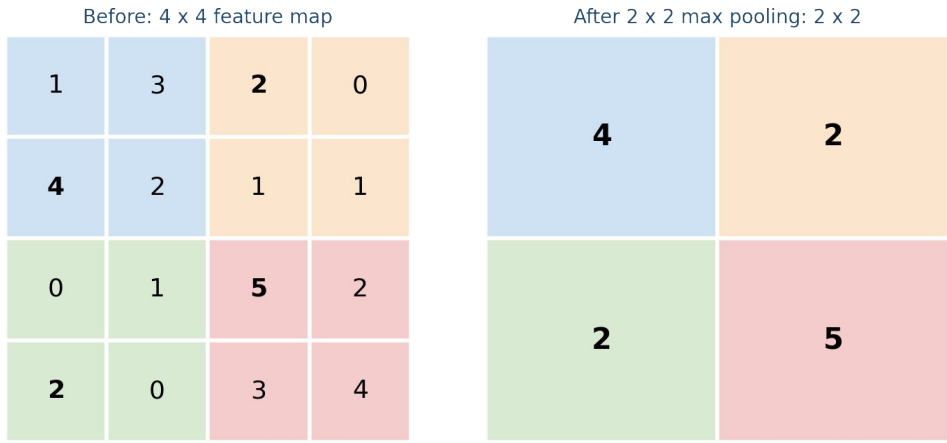
A 3 x 3 filter shrinks the image unless we add a border of zeros (padding). Stride is how many pixels the filter moves each step. In Keras, padding='valid' means no border and padding='same' keeps the size.

Setting	Meaning	Output from a 48 x 48 image (3 x 3 filter)
padding 0 ('valid'), stride 1	No border, move one pixel at a time	46 x 46 (the image shrinks)
padding 1 ('same'), stride 1	Border of zeros, size kept	48 x 48 (same as the input)
padding 0, stride 2	No border, jump two pixels at a time	23 x 23
padding 1, stride 2	Border, and jump two at a time	24 x 24
Formula	n = image size, k = filter size, p = padding, s = stride	output = (n + 2p - k) / s + 1, rounded down

CONCEPT

Pooling: Shrink, Keep the Strongest

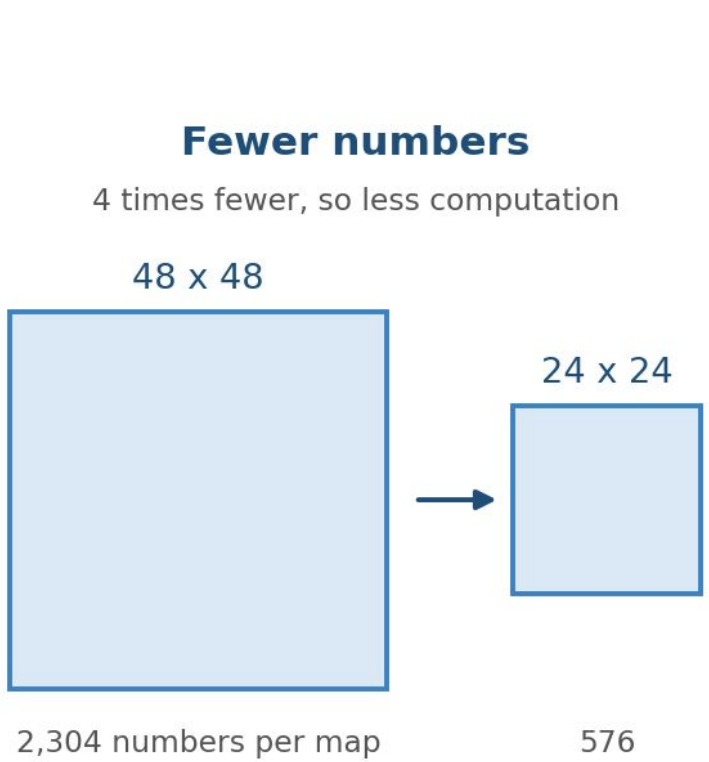
- Max pooling looks at each 2 x 2 block and keeps only the largest value
- The map halves in each direction (48 x 48 becomes 24 x 24), so later layers have 4 times fewer numbers to handle
- It also makes the network tolerant of small shifts: if a feature moves by one pixel, the strongest value in the block is often the same
- Pooling has no weights to learn. It is a fixed rule

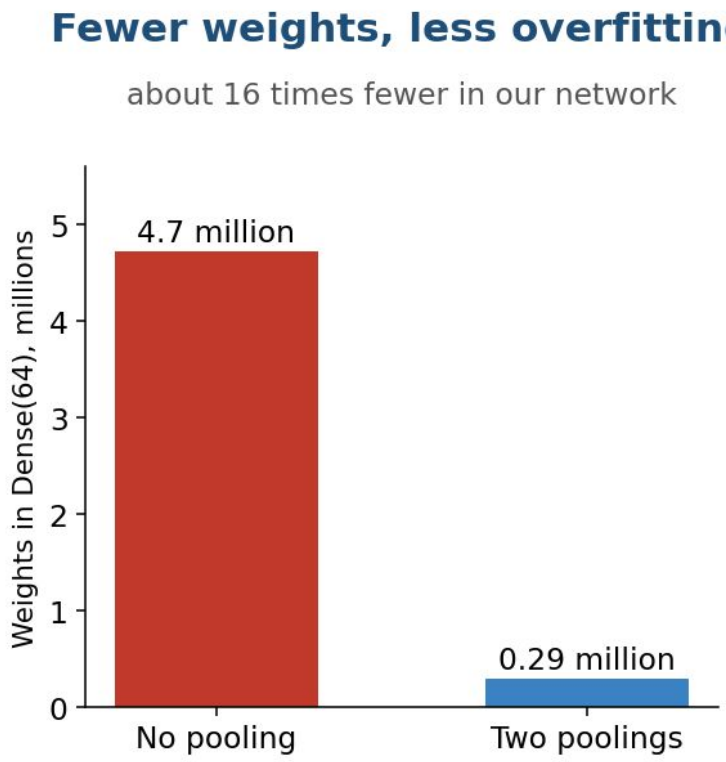


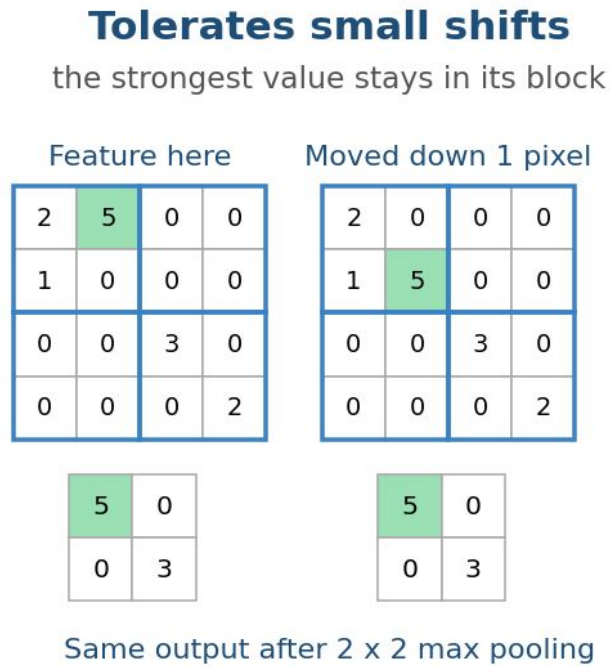
CONCEPT

Benefits of Pooling

Pooling shrinks every map, which cuts computation and weights, and makes the result less sensitive to exact position.







15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

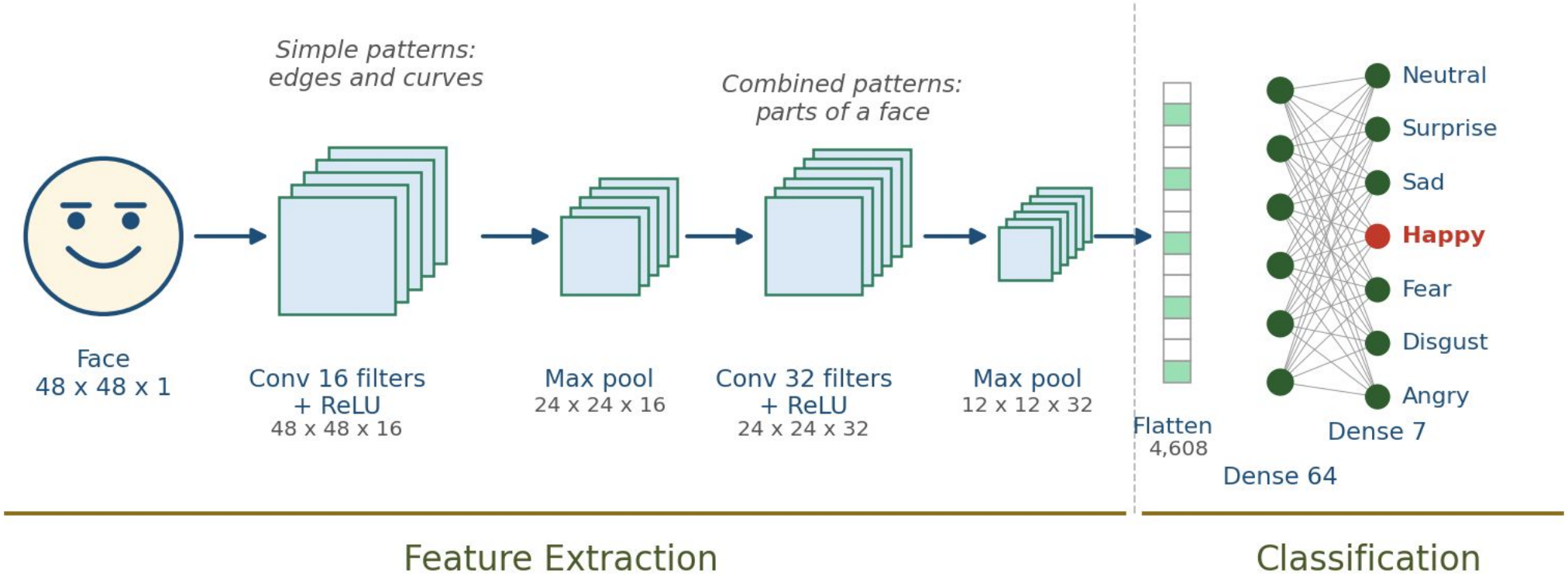
Hour 3

Building EmotionCNN in TensorFlow

CONCEPT

The Whole Network

Convolution finds patterns, ReLU adds non-linearity, pooling shrinks the map, and we use this block twice. Keras tensors are (batch, height, width, channels), with channels last.



EXAMPLE

Checking the Shapes

Before building the model, watch how the tensor shape changes. Shape errors are the most common CNN bug.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

x = tf.zeros((1, 48, 48, 1))                  # 1 image, 48 x 48 pixels, 1 channel
conv1 = layers.Conv2D(16, 3, padding='same', activation='relu')
conv2 = layers.Conv2D(32, 3, padding='same', activation='relu')
pool = layers.MaxPooling2D(2)

a = conv1(x);  print(a.shape)                 # (1, 48, 48, 16)   16 feature maps
b = pool(a);   print(b.shape)                 # (1, 24, 24, 16)   each map halved
c = conv2(b);  print(c.shape)                 # (1, 24, 24, 32)   32 feature maps
d = pool(c);   print(d.shape)                 # (1, 12, 12, 32)   halved again
print(layers.Flatten()(d).shape)              # (1, 4608)         12 x 12 x 32 = 4,608
```

EXAMPLE

Defining EmotionCNN

Keras Sequential stacks the layers in order. Conv2D takes the number of filters first, then the filter size.

```
keras.utils.set_random_seed(42)              # same starting weights every run

model = keras.Sequential([
    layers.Input(shape=(48, 48, 1)),
    layers.Conv2D(16, 3, padding='same', activation='relu'),   # 16 filters of 3 x 3
    layers.MaxPooling2D(2),
    layers.Conv2D(32, 3, padding='same', activation='relu'),   # 32 filters of 3 x 3
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(7),                                           # 7 raw scores, one per emotion
])
model.summary()          # Total params: 300,231
```

CONCEPT

EmotionCNN, Layer by Layer

Counting parameters shows where the weights live. Note that almost all of them are in the first Dense layer.

Layer	Output shape	Parameters	What it does
Input	48 x 48 x 1	0	One grayscale face
Conv2D(16, 3, same)	48 x 48 x 16	16 x 3 x 3 + 16 = 160	Sixteen 3 x 3 pattern detectors
MaxPooling2D(2)	24 x 24 x 16	0	Halves each map
Conv2D(32, 3, same)	24 x 24 x 32	32 x 3 x 3 x 16 + 32 = 4,640	Thirty-two detectors on the 16 maps
MaxPooling2D(2)	12 x 12 x 32	0	Halves each map again
Flatten	4,608	0	Unrolls the maps into a list
Dense(64)	64	4,608 x 64 + 64 = 294,976	Combines all the features
Dense(7)	7	64 x 7 + 7 = 455	One score per emotion
Total		300,231	All learned by gradient descent

QUICK CHECK

Quick Check

What would change in the model if we used padding='valid' instead of padding='same' in both convolutions? Would the code still run?

ANSWER

Quick Check: Answer

```
# The maps shrink at every convolution: 48 -> 46, pool -> 23,
# then 23 -> 21, pool -> 10. Flatten gives 10 x 10 x 32 = 3,200
# numbers instead of 4,608.
#
# The code still runs, because Keras works out the input size of
# Dense(64) by itself (PyTorch's Linear needs it written out).
# The model gets smaller: Dense(64) has 3,200 x 64 + 64 = 204,864
# weights, and the whole model 210,119 instead of 300,231.
```

CONCEPT

A Loss for Seven Classes

- VillainNet had one output and a yes or no answer. EmotionCNN has 7 outputs, so we need a loss built for choosing among classes
- SparseCategoricalCrossentropy takes the 7 raw scores and the true emotion number (0 to 6). Internally it turns the scores into probabilities (softmax) and punishes confident wrong answers
- from_logits=True tells Keras that the scores are raw. That is why the last Dense layer has no activation
- 'Sparse' means the labels are plain numbers like 3, not one-hot lists like [0, 0, 0, 1, 0, 0, 0]. Everything else from Day 20 is the same: Adam, forward pass, loss, backward pass

EXAMPLE

Training EmotionCNN

compile() chooses the optimizer, the loss and what to report. fit() runs the training loop, now on mini-batches of 64 faces, and also checks the validation set after every epoch.

```
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
              loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(X_train, y_train,
                    validation_data=(X_val, y_val),
                    epochs=15, batch_size=64)
# Each epoch prints loss, accuracy, val_loss and val_accuracy.
# Turn on the GPU accelerator (Day 20) to make this faster.
```

EXAMPLE

Reading the Loss Curve

Plot what fit() recorded. Before you run it, predict the shape: where do you expect training and validation loss to part ways?

```
plt.plot(history.history['loss'], label='training loss')
plt.plot(history.history['val_loss'], label='validation loss')
plt.xlabel('epoch')
plt.ylabel('loss')
plt.legend()
plt.show()

print(max(history.history['val_accuracy']))      # the best validation accuracy reached
```

CONCEPT

What to Look For in the Curves

- Training loss falls steadily. That is the network getting better at the faces it has seen
- Validation loss falls at first, then flattens or starts to rise while training loss keeps falling. That gap is overfitting, the same warning sign as Day 19
- The epoch where validation loss is lowest is the best place to stop. Early stopping, later today, does this automatically
- A model that is only slightly better than random guessing (about 14 percent for 7 classes) after many epochs has a problem. Check that the pixels were scaled and the labels match

EXAMPLE

Evaluating on the Test Set

The test set is used once, at the end. predict() returns raw scores, and argmax picks the highest.

```
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f'Test accuracy: {test_acc:.3f}')

scores = model.predict(X_test, verbose=0)         # raw scores, shape (3589, 7)
predictions = scores.argmax(axis=1)               # the highest of the 7 scores wins
print((predictions == y_test).mean())             # the same accuracy, computed by hand
```

EXAMPLE

Where Does It Go Wrong?

A confusion matrix shows which emotions get mixed up. Each row is the true emotion and each column the prediction.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report

cm = confusion_matrix(y_test, predictions)       # rows = true emotion, columns = predicted
ConfusionMatrixDisplay(cm, display_labels=EMOTIONS).plot(xticks_rotation=45)
plt.show()

print(classification_report(y_test, predictions, target_names=EMOTIONS))
```

CONCEPT

Reading the Confusion Matrix

- The diagonal is correct answers. Everything off the diagonal is a mistake, and the biggest off-diagonal numbers show which emotions the network confuses
- Look at the recall of each emotion in the report. Rare emotions such as Disgust usually score far below common ones such as Happy, because the network sees fewer examples
- Accuracy alone hides this. A model can reach a decent overall score while almost never predicting the rare classes
- Ask yourself: would a person confuse the same pairs? Fear and Surprise look alike even to us

EXAMPLE

Try It on One Face

Softmax turns the 7 raw scores into probabilities that add up to 1, so we can see how confident the network is.

```
i = 0                                          # try any test face: 0, 1, 2 ...
probs = tf.nn.softmax(scores[i]).numpy()       # raw scores -> probabilities that add up to 1

plt.imshow(X_test[i].reshape(48, 48), cmap='gray'); plt.axis('off'); plt.show()
print('Actual:   ', EMOTIONS[y_test[i]])
print('Predicted:', EMOTIONS[probs.argmax()], round(float(probs.max()), 2))
for name, p in zip(EMOTIONS, probs):
    print(f'{name:9s} {p:.2f}')
```

EXAMPLE

What Did the Filters Learn?

The 16 filters of the first layer, as small images. We never told the network to look for anything in particular.

```
filters = model.layers[0].get_weights()[0]       # first Conv2D layer: weights, not biases
print(filters.shape)                             # (3, 3, 1, 16)   16 filters of 3 x 3

fig, axes = plt.subplots(2, 8, figsize=(12, 3.2))
for k, ax in enumerate(axes.ravel()):
    ax.imshow(filters[:, :, 0, k], cmap='coolwarm')    # blue = positive, red = negative
    ax.axis('off')
plt.show()
```

CONCEPT

Reading the Filters

- Blue cells are positive weights and red cells negative: each filter responds to a different small pattern, such as an edge in a certain direction
- After training, some filters look like clear edge detectors and others look noisy. Noisy ones are normal in a small first layer
- We never designed these. The network discovered useful features itself, which is why CNNs took over image tasks
- Compare your filters with a classmate's: the same code with a different random start gives different filters

EXAMPLE

CNN vs. a Dense Network

A fair test: a dense network with about the same number of weights (303,751 against 300,231), trained the same way.

```
dense = keras.Sequential([
    layers.Input(shape=(48, 48, 1)),
    layers.Flatten(),                              # layout thrown away: 2,304 numbers
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(7),
])
dense.compile(optimizer='adam', metrics=['accuracy'],
              loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True))
dense.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=15, batch_size=64)

print(dense.count_params())                        # 303,751   about the same size as the CNN
print(dense.evaluate(X_test, y_test, verbose=0)[1])    # its test accuracy: compare with the CNN
```

EXAMPLE

The Shift Test

Slide every test face 2 pixels to the right and score both models again. No retraining.

```
X_shift = np.roll(X_test, shift=2, axis=2)       # slide every face 2 pixels right (axis 2 =
width)

print('CNN  :', model.evaluate(X_shift, y_test, verbose=0)[1])
print('Dense:', dense.evaluate(X_shift, y_test, verbose=0)[1])
# Which model loses more accuracy when the faces move? Write down both numbers.
```

CONCEPT

Why the CNN Should Cope With Shifts

A dense network learns one weight per pixel position, so a shifted face looks like a new input. A CNN slides the same filter everywhere, so the response just moves with the face. Compare the size of the drop in your own test, not the exact value.

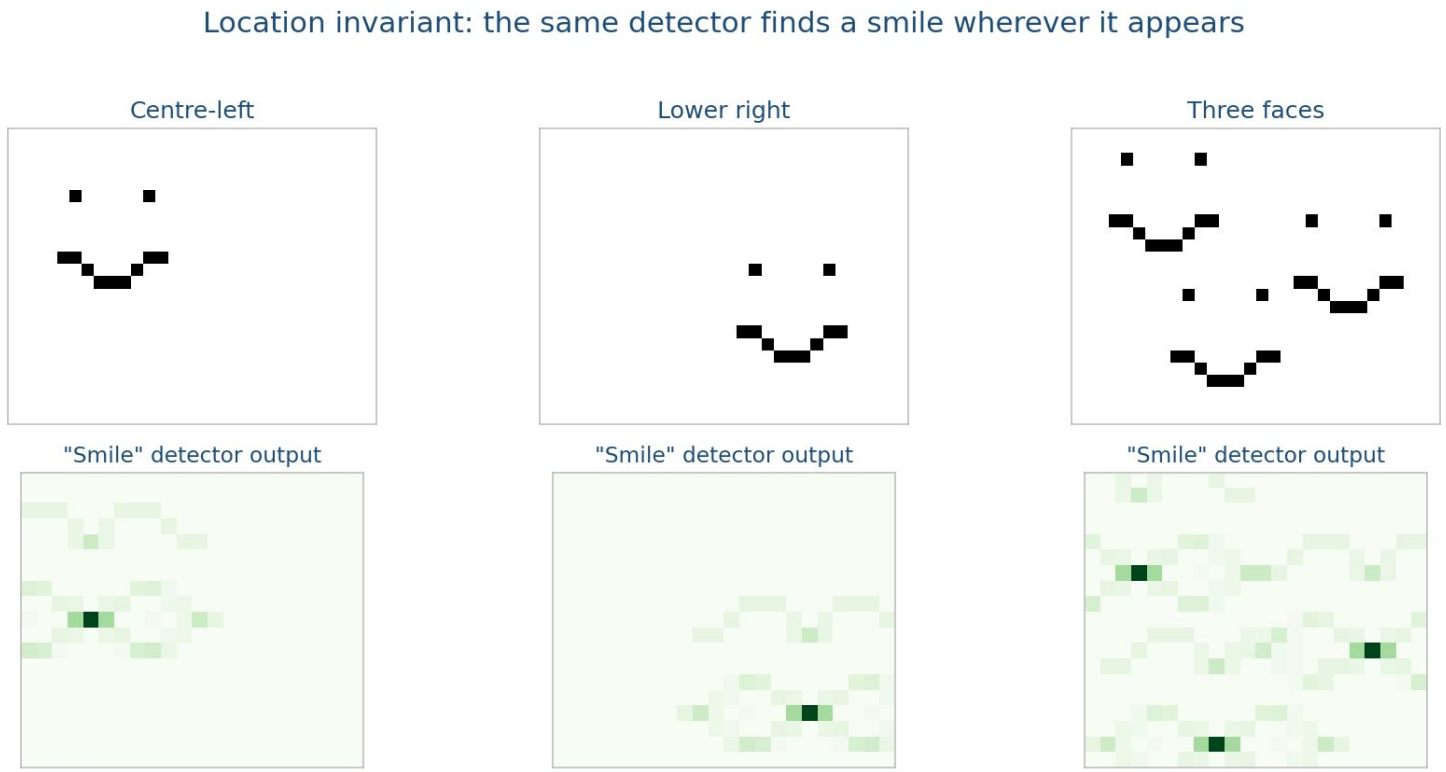


Illustration: simplified faces. The bright spots are real outputs of one 6 x 9 smile filter slid over each picture.

PROJECT

Ideas to Improve the Score

- FER-2013 is hard, so a small CNN will not be perfect. Each idea below is a separate experiment: change one thing at a time, so you know what helped
- Data augmentation: show the network randomly flipped, rotated and zoomed faces, so it sees more variety than 28,709 fixed images
- BatchNormalization and Dropout: steadier training and less overfitting
- Class weights: make mistakes on rare emotions cost more. Callbacks: stop early and lower the learning rate when progress stalls
- Transfer learning (an extension): start from a network already trained on millions of photos, such as MobileNetV2, resize the faces to 96 x 96 and repeat the grey channel three times

EXAMPLE

Idea 1: Data Augmentation

These layers create new versions of each face on the fly. They are active only during training.

```
augment = keras.Sequential([
    layers.RandomFlip('horizontal'),            # a mirrored face has the same emotion
    layers.RandomRotation(0.1),                 # turn by up to 36 degrees
    layers.RandomZoom(0.1),                     # zoom in or out by up to 10 percent
])
# These layers change the image only while training. At test time they do nothing.
# Place augment right after Input(...) in your model, then compare validation accuracy.
```

EXAMPLE

Idea 2: BatchNorm and Dropout

A second model with the augmentation inside it. Compile it the same way as before.

```
model2 = keras.Sequential([
    layers.Input(shape=(48, 48, 1)),
    augment,
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),                  # steadier training
    layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dropout(0.5),                          # switch off half the values
    layers.Dense(128, activation='relu'),
    layers.Dense(7),
])
model2.compile(optimizer='adam', metrics=['accuracy'],
               loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

EXAMPLE

Idea 3: Class Weights and Callbacks

compute_class_weight gives rare classes larger weights. EarlyStopping and ReduceLROnPlateau watch the validation loss.

```
from sklearn.utils.class_weight import compute_class_weight

weights = compute_class_weight('balanced', classes=np.arange(7), y=y_train)
class_weight = dict(enumerate(weights))          # rare emotions weigh more

callbacks = [
    keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True),   # stop when stalled
    keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=2),              # halve the rate
]
history2 = model2.fit(X_train, y_train, validation_data=(X_val, y_val),
                      epochs=40, batch_size=64,
                      class_weight=class_weight, callbacks=callbacks)
```

PROJECT

Your Experiment Log

Fill this in as you run each version. Change one thing at a time, and note what helped and what did not.

Experiment	What changed	Validation accuracy	Test accuracy
1. Baseline EmotionCNN	Nothing		
2. Dense network	No convolutions		
3. Add augmentation	RandomFlip, RandomRotation, RandomZoom		
4. Add BatchNorm and Dropout	model2		
5. Add class weights and callbacks	class_weight, EarlyStopping		
6. Your own idea			

CONCEPT

Emotion Recognition: Handle With Care

- A model that outputs "Angry" has not seen anger. It has matched pixel patterns to labels that other people chose, and those labels are often debatable
- Facial expressions do not reliably show what someone feels, and they differ across cultures and between people. A model trained on one set of faces may fail on others
- Faces are personal data. Filming people and judging their emotions, for example at work or in a classroom, raises privacy and fairness questions
- Use models like this for learning and for low-stakes ideas, never to judge a real person. This links to the bias warning on Day 20

CONCEPT

Where CNNs Are Used

- Reading number plates, cheques, forms and handwritten slips
- Medical imaging: spotting problems in X-rays and scans
- Agriculture: finding diseased leaves on a crop from a phone photo
- Quality control: spotting defects on a factory line, and checking photos of products on a marketplace
- The same filter-and-pool idea also works on audio spectrograms and other grid-shaped data

CONCEPT

Habits Worth Keeping

- Always check shapes after each layer. Print them or call model.summary() while building
- Keras image tensors are (batch, height, width, channels). PyTorch puts the channels before the height and width, so watch out when you move between libraries
- Scale pixel values before training. Look at your data before you model it
- Start with a small CNN and a baseline, then improve one thing at a time and keep a log

PRACTICE

Practice

A teammate says: "CNNs are just dense networks that got lucky on images." Using today's experiments, give two concrete reasons why a CNN is a better fit for images than a dense network.

SUMMARY

Key Takeaways

- An image is a grid of numbers: height x width, with one channel for grayscale and three for colour
- A convolution slides a small learned filter over the image to detect a pattern; pooling shrinks the result and adds tolerance to shifts
- A CNN shares weights across the image, so it can match a dense network of the same size while being built for images
- In TensorFlow a CNN is a Sequential model: Conv2D, MaxPooling2D, Flatten and Dense, trained with compile() and fit()
- Improve one thing at a time and keep a log. Next up: the Deep Learning mini-project